

مستند طراحی پایگاه داده سامانه بانکی

(1) مقدمه

امروزه سامانه‌های اطلاعاتی بانکی به عنوان یکی از حساس‌ترین و مهم‌ترین سامانه‌های نرم‌افزاری شناخته می‌شوند، زیرا با داده‌هایی سروکار دارند که هم از نظر مالی و هم از نظر امنیتی بسیار ارزشمند هستند. در چنین سیستم‌هایی، پایگاه داده نقش ستون فقرات را ایفا می‌کند و تمام عملیات اصلی مانند ثبت مشتری، افتتاح حساب، مدیریت کارت، ثبت تراکنش، صدور چک، اعطای وام، مدیریت شعب، نگهداری دستگاه‌ها و همچنین ثبت سوابق تغییرات، بر پایه طراحی درست پایگاه داده انجام می‌شود. یک پایگاه داده خوب باید چند ویژگی اصلی داشته باشد:

- درستی و انسجام داده‌ها
- جلوگیری از تکرار غیرضروری اطلاعات
- امکان توسعه و نگهداری آسان
- پشتیبانی از امنیت و سطح دسترسی
- سرعت مناسب در جست‌وجو و گزارش‌گیری
- قابلیت ثبت تاریخچه و ردیابی تغییرات

پایگاه داده به‌گونه‌ای طراحی شده است که بخش‌های مختلف یک سامانه بانکی را پوشش دهد. این طراحی فقط به ذخیره اطلاعات خام محدود نشده، بلکه برای عملیات واقعی کسب‌وکار نیز منطق در نظر گرفته شده است؛ مانند کنترل نوع تراکنش، ثبت تاریخچه تغییر موجودی حساب، مدیریت مالکیت حساب، نگهداری اطلاعات مشتری حقیقی و حقوقی، و اعمال محدودیت‌های امنیتی و دسترسی.

(2) اصول طراحی پایگاه داده

طراحی پایگاه داده فقط ساخت چند جدول نیست، بلکه یک فرایند مهندسی‌شده برای سازمان‌دهی اطلاعات است. در طراحی صحیح، باید چند اصل مهم رعایت شود:

(2-1) تفکیک منطقی داده‌ها

هر موجودیت باید در جدول مناسب خود ذخیره شود. برای مثال:

- اطلاعات پایه مشتری در یک جدول

• اطلاعات مشتری حقیقی در جدول جدا

• اطلاعات مشتری حقوقی در جدول جدا

• اطلاعات حساب در جدول مستقل

• روابط مالکیت بین مشتری و حساب در جدول واسط

این روش باعث می‌شود داده‌ها منظم‌تر، قابل‌فهم‌تر و قابل‌توسعه‌تر باشند.

(2-2) کاهش افزونگی

افزونگی یعنی ذخیره اطلاعات تکراری در چند نقطه. اگر این کار کنترل نشود، احتمال ناسازگاری داده‌ها بالا می‌رود. در این پروژه، با استفاده از ساختارهای نرمال‌شده و جدول‌های مرتبط، تلاش شده است که اطلاعات در یک محل مشخص نگهداری شوند و از تکرار غیرضروری جلوگیری شود.

(2-3) حفظ یکپارچگی داده‌ها

یکپارچگی یعنی داده‌های ذخیره‌شده با قوانین سیستم سازگار باشند.

برای نمونه:

• تاریخ تولد نباید در آینده باشد

• سن مشتری حقیقی باید حداقل ۱۸ سال باشد

• نوع تراکنش باید با ساختار آن هماهنگ باشد

• مانده حساب نباید بدون ثبت تاریخچه تغییر کند

در این پایگاه داده، چنین محدودیت‌هایی به‌صورت جدی در طراحی در نظر گرفته شده‌اند.

(2-4) امنیت

در سامانه بانکی، امنیت از مهم‌ترین موضوعات است. بنابراین:

• رمز عبور و برخی داده‌های حساس به‌صورت هش‌شده نگهداری می‌شوند

• نقش‌ها و سطح دسترسی‌ها کنترل می‌شوند

• داده‌ها در سطح دیتابیس نیز رمزنگاری می‌شوند

• عملیات حذف برای بعضی نقش‌ها محدود شده است

(2-5) قابلیت گزارش‌گیری

یک پایگاه داده بانکی باید بتواند گزارش‌های مدیریتی و عملیاتی تولید کند.

در این پروژه، علاوه بر جداول اصلی، viewها، functionها و queryهای تحلیلی نیز طراحی شده‌اند تا بتوان:

- وضعیت مشتریان را بررسی کرد
- وضعیت حساب‌ها را دید
- تراکنش‌ها را تحلیل کرد
- موجودی ATMها را کنترل کرد
- وضعیت وام‌ها و چک‌ها را گزارش کرد

(2-6) قابلیت ردیابی تغییرات

در سیستم‌های مالی، دانستن اینکه چه چیزی، چه زمانی و چگونه تغییر کرده بسیار مهم است.

برای همین، history tables و triggerها در طراحی استفاده شده‌اند تا تغییرات مهم مانند:

- تغییر مانده حساب
- تغییر وضعیت تراکنش

در سوابق ثبت شوند.

(3) آشنایی با ER و EER

(3-1) مدل ER چیست؟

مدل ER یا Entity-Relationship یکی از روش‌های پایه برای طراحی پایگاه داده است. در این مدل، ابتدا موجودیت‌ها، ویژگی‌ها و ارتباط بین آن‌ها شناسایی می‌شوند.

مفاهیم اصلی ER:

• **Entity (موجودیت):** چیزی که اطلاعات درباره آن ذخیره می‌شود؛ مثل مشتری، حساب، کارت، شعبه

• **Attribute (ویژگی):** خصوصیات هر موجودیت؛ مثل نام، ایمیل، شماره حساب، تاریخ افتتاح

• **Relationship (رابطه):** ارتباط بین موجودیت‌ها؛ مثل مالکیت مشتری بر حساب

(3-2) چرا ER مهم است؟

مدل ER کمک می‌کند قبل از پیاده‌سازی فنی، ساختار مفهومی سیستم مشخص شود.

به این ترتیب:

- موجودیت‌ها درست شناسایی می‌شوند

- ارتباط‌ها شفاف می‌شوند

- طراحی جدول‌ها منطقی‌تر می‌شود

3-3 مدل EER چیست؟

مدل EER یا Enhanced ER نسخه پیشرفته‌تر ER است.

این مدل برای سیستم‌هایی استفاده می‌شود که موجودیت‌های پیچیده‌تری دارند یا نیاز به تخصص‌بخشی دارند.

مفاهیم اضافه در EER:

- **Generalization / Specialization**: مثلاً یک customer می‌تواند به customer حقیقی و customer

حقوقی تخصص پیدا کند

- **Inheritance**: موجودیت فرزند ویژگی‌های موجودیت والد را به ارث می‌برد

- **Subclass / Superclass**: برای جداسازی ویژگی‌های مشترک و اختصاصی

3-4 کاربرد EER در این پروژه

ساختار این پایگاه داده به‌وضوح از منطق EER استفاده می‌کند.

برای مثال:

- یک customer به‌عنوان موجودیت پایه در نظر گرفته شده

- سپس به دو زیرنوع تقسیم شده:

- **individual_customer**

- **organization_customer**

این دقیقاً یک نمونه از **specialization** است.

یعنی اطلاعات عمومی در جدول پایه مشتری قرار گرفته و اطلاعات اختصاصی هر نوع مشتری در جدول فرزند ذخیره می‌شود.

3-5 مزایای این رویکرد

- تفکیک بهتر داده‌های مشترک و اختصاصی

- کاهش تکرار

- افزایش خوانایی طراحی
- مناسب برای سامانه‌های بزرگ و چندنوعی مانند بانک‌ها

4) مدل رابطه‌ای Relational Model

4-1) تعریف مدل رابطه‌ای

مدل رابطه‌ای یکی از رایج‌ترین مدل‌ها در پایگاه داده است. در این مدل، داده‌ها در قالب **جدول** ذخیره می‌شوند. هر جدول شامل:

• **سطرها** یا رکوردها

• **ستون‌ها** یا فیلدها

4-2) اجزای اصلی مدل رابطه‌ای

• **Relation**: همان جدول

• **Tuple**: هر سطر از جدول

• **Attribute**: هر ستون از جدول

• **Primary Key**: کلید اصلی برای شناسایی یکتای هر رکورد

• **Foreign Key**: کلید خارجی برای ایجاد ارتباط بین جدول‌ها

4-3) اهمیت مدل رابطه‌ای در این پروژه

کل پایگاه داده طراحی‌شده بر پایه مدل رابطه‌ای ساخته شده است.

جدول‌ها به وسیله کلیدهای اصلی و خارجی به هم متصل هستند.

برای مثال:

• مشتری به حساب از طریق جدول واسط مالکیت متصل می‌شود

• حساب به کارت متصل است

• حساب به وام و چک‌بوک نیز مرتبط است

• تراکنش‌ها به حساب مبدأ، حساب مقصد و دستگاه مبدأ متصل‌اند

4-4) مزایای مدل رابطه‌ای

• سادگی در درک و پیاده‌سازی

- پشتیبانی خوب از یکپارچگی داده
- مناسب برای کوئری‌های پیچیده
- توانایی بالای گزارش‌گیری
- سازگاری با اصول نرمال‌سازی

(5) معرفی کلی پایگاه داده

این پایگاه داده یک **سامانه بانکی چندبخشی** است که موجودیت‌های اصلی آن شامل موارد زیر می‌شوند:

- مشتری
- شعبه
- کارمند
- ارز
- حساب
- دستگاه
- کارت
- ATM
- تراکنش
- چک
- وام
- پیام
- تاریخچه‌های حسابرسی
- نقش‌ها و دسترسی‌ها

دیتابیس نشان می‌دهد که سیستم فقط برای نگهداری اطلاعات پایه نیست.

به همین دلیل، علاوه بر جداول اصلی، بخش‌هایی مانند:

- stored procedure
- function
- trigger
- index
- RBAC

- encryption

نیز در نظر گرفته شده‌اند.

6) اجزای اصلی پایگاه داده

6-1) مشتریان

6-1-1) جدول پایه مشتری

این جدول، هسته اصلی بخش مشتریان است و اطلاعات عمومی همه مشتری‌ها را نگه می‌دارد. اطلاعاتی مانند:

- شماره تماس
 - ایمیل
 - رمز عبور هش شده
 - وضعیت حساب کاربری
 - نوع مشتری
 - زمان ایجاد و به‌روزرسانی
- در این جدول نگهداری می‌شود.

نقش این جدول

این جدول نقش مرکزی دارد، چون همه انواع مشتری از آن منشعب می‌شوند.

6-1-2) مشتری حقیقی

این بخش مخصوص افراد حقیقی است و اطلاعات هویتی آن‌ها را نگه می‌دارد:

- نام
- نام خانوادگی
- تاریخ تولد
- کد/شناسه هویتی
- شغل
- آدرس

قوانین مهم

• تاریخ تولد باید معتبر باشد

• سن مشتری باید حداقل ۱۸ سال باشد

این محدودیت‌ها برای اعتبارسنجی هویتی و قانونی بسیار مهم هستند.

6-1-3 مشتری حقوقی

این بخش برای شرکت‌ها و سازمان‌ها طراحی شده است.

اطلاعاتی مانند:

• نام سازمان

• شماره ثبت

• تاریخ تأسیس

• صنعت یا حوزه فعالیت

• آدرس مرکزی

• شخص رابط

• شناسه مدیرعامل یا نماینده

در این بخش ذخیره می‌شود.

نکته مهم

در طراحی پایگاه داده نام شرکت ارتباط انسانی میان سازمان و بانک را نیز لحاظ شده است.

6-2 شعبه

جدول شعبه برای نگهداری اطلاعات واحدهای فیزیکی بانک استفاده می‌شود.

اطلاعاتی مانند:

• کد شعبه

• نام شعبه

• شهر

• آدرس

• تاریخ تأسیس

در آن ذخیره می‌شود.

کاربرد شعبه در سیستم

شعبه در چند بخش مهم استفاده می‌شود:

- انتساب کارمندان

- صدور برخی خدمات

- ثبت چک‌بوک

- مدیریت ATM‌ها

بنابراین، شعبه یکی از موجودیت‌های زیرساختی مهم در سیستم است.

3-6) کارمندان

جدول کارمندان اطلاعات کارکنان بانک را ذخیره می‌کند.

اطلاعاتی مانند:

- نام و نام خانوادگی

- کد هویتی

- ایمیل

- تلفن

- رمز عبور

- نقش شغلی

- آدرس

- تاریخ استخدام

- تاریخ پایان همکاری

- شعبه مربوطه

نقش کارمندان

کارمندان ممکن است شامل مسئول باجه، مدیر، یا سایر نقش‌های داخلی باشند.

سطح سازمانی کاربران نیز در طراحی لحاظ شده است.

4-6) ارزش

جدول ارزش برای نگهداری واحدهای پولی استفاده می‌شود.

این جدول کمک می‌کند تا حساب‌ها و موجودی ATM‌ها بر اساس ارزش طبقه‌بندی شوند.

اهمیت جدول ارزش

- پشتیبانی از چند ارزش
- تفکیک حساب‌ها بر اساس واحد پول
- گزارش‌گیری مالی دقیق‌تر

5-6) حساب

جدول حساب یکی از مهم‌ترین اجزای پایگاه داده است.

اطلاعاتی مانند:

- شماره حساب
- نوع حساب
- موجودی
- وضعیت حساب
- ارزش حساب
- تاریخ بسته شدن

در این جدول ذخیره می‌شود.

وضعیت‌های حساب

حساب می‌تواند در وضعیت‌های مختلف باشد، از جمله:

- فعال
- مسدود
- غیر فعال
- بسته شده

نوع‌های حساب

در سیستم، چند نوع حساب قابل مشاهده است:

- پس‌انداز
- جاری
- تجاری
- وام
- سرمایه‌گذاری

نقش حساب

تمام عملیات مالی به‌نوعی به حساب متصل هستند:

- مالکیت حساب
- کارت
- وام
- چک‌بوک
- تراکنش‌ها

6-6 مالکیت حساب

از آنجا که یک حساب ممکن است بیش از یک مالک داشته باشد، رابطه مشتری و حساب در یک جدول واسطه نگهداری می‌شود.

این ساختار چه مزیتی دارد؟

- امکان چندمالکی برای حساب
- مدیریت نوع مالکیت
- تفکیک مالک اصلی و مالکان فرعی
- کنترل بهتر روی عملیات حذف و تغییر مالکیت

قوانین مهم

- مالک اصلی حساب را نمی‌توان به‌سادگی حذف کرد
- تغییر نوع مالکیت نیز با بررسی تعداد مالکان انجام می‌شود

منطق کسب و کاری در سطح پایگاه داده لحاظ شده است.

6-7) دستگاه

جدول دستگاه برای ثبت دستگاه‌های متصل به مشتری استفاده می‌شود. این موجودیت می‌تواند برای مدیریت دسترسی، ثبت دستگاه‌های مجاز یا استفاده در تراکنش‌ها به کار رود.

نقش دستگاه در سامانه

- ثبت دستگاه‌های کاربر
- استفاده در منبع برخی تراکنش‌ها
- پشتیبانی از کنترل امنیتی بیشتر

6-8) کارت

جدول کارت برای نگهداری کارت‌های بانکی استفاده می‌شود. کارت‌ها به حساب متصل‌اند و اطلاعاتی مانند:

- شماره کارت
- حساب وابسته
- تاریخ انقضا
- وضعیت کارت
- CVV رمزنگاری شده

را شامل می‌شوند.

اهمیت کارت

کارت یکی از کانال‌های اصلی دسترسی به حساب است و در گزارش‌ها و queryها نیز مورد استفاده قرار گرفته است.

6-9) ATM

بخش ATM برای مدیریت دستگاه‌های خودپرداز طراحی شده است.

اطلاعاتی مانند:

• شعبه مربوطه

• شهر

• آدرس

• تاریخ تأسیس

در این بخش ذخیره می‌شود.

ATM cash

علاوه بر خود ATM، موجودی نقدی آن نیز بر حسب ارز ذخیره می‌شود.

این موضوع برای کنترل عملکرد عملیاتی دستگاه‌ها بسیار مهم است.

6-10 تراکنش‌ها

جدول تراکنش یکی از حساس‌ترین جداول سیستم است.

اطلاعاتی مانند:

• نوع تراکنش

• مبلغ

• وضعیت

• توضیح

• حساب مبدأ

• حساب مقصد

• دستگاه مبدأ

• زمان صدور

• زمان تکمیل

در آن ذخیره می‌شود.

انواع تراکنش

در طراحی این سیستم تراکنش‌هایی مانند موارد زیر وجود دارند:

• انتقال

- برداشت
- واریز
- خرید
- پرداخت قبض

ویژگی مهم

ساختار جدول تراکنش با محدودیت‌های جدی همراه است تا نوع تراکنش با نوع حساب‌های درگیر سازگار باشد.

برای مثال:

- برداشت باید حساب مبدأ داشته باشد
 - واریز باید حساب مقصد داشته باشد
 - انتقال باید هر دو حساب مبدأ و مقصد را داشته باشد
 - پرداخت قبض نیز ساختار مخصوص خود را دارد
- توجه به منطق واقعی عملیات بانکی است.

6-11) چک‌بوک و چک

چک‌بوک

چک‌بوک برای یک حساب و یک شعبه صادر می‌شود و شامل:

- تاریخ صدور
- تعداد برگه‌ها
- وضعیت

است.

چک

هر چک دارای اطلاعاتی مانند:

- شماره چک
- چک‌بوک مربوطه
- حساب دریافت‌کننده

- مبلغ
- تاریخ صدور
- تاریخ انقضا
- تاریخ وصول
- وضعیت

می باشد.

نقش این بخش

عملیات چک در پایگاه داده به صورت کامل و منظم پشتیبانی شده است، از سطح صدور چک بک تا ثبت وضعیت تک برگ های چک.

6-12) وام

جدول وام اطلاعات تسهیلات اعطایی را نگهداری می کند:

- حساب مرتبط
- مشتری ضامن
- مبلغ وام
- نرخ بهره
- مدت وام
- وضعیت بازپرداخت
- تاریخ صدور

وضعیت های وام

- فعال
- تسویه شده
- نکول شده

اهمیت

این جدول امکان تحلیل اعتباری، مدیریت مطالبات و بررسی تعهدات مالی را فراهم می کند.

13-6) پیام‌ها

جدول پیام برای ذخیره پیام‌های مشتریان استفاده می‌شود.

اطلاعاتی مانند:

- موضوع
- متن پیام
- وضعیت خوانده شدن
- تاریخ ایجاد
- تاریخ خوانده شدن

در آن ذخیره می‌شود.

کاربرد

این بخش برای ارتباط بین بانک و مشتری بسیار مفید است و نشان می‌دهد سیستم فقط مالی نیست، بلکه جنبه ارتباطی نیز دارد.

14-6) تاریخچه و حسابرسی

دیتابیس برای نگه داری بیشتر اطلاعات نیازمند وجود جداول تاریخچه است.

تاریخچه مانده حساب

هر بار که مانده حساب تغییر می‌کند، نسخه‌ای از مقدار قبلی و جدید ذخیره می‌شود.

تاریخچه وضعیت تراکنش

هر بار وضعیت تراکنش تغییر کند، این تغییر نیز ثبت می‌شود.

مزایای این کار

- قابلیت audit
- ردیابی تغییرات
- بررسی خطاها
- انطباق با الزامات امنیتی و مالی

Full schema

```
;CREATE DATABASE BankSystemV1
```

```
GO
```

```
USE BankSystemV1;
```

```
GO
```

```
/* =====  
SCHEMAS  
===== */
```

```
CREATE SCHEMA customer;
```

```
GO
```

```
CREATE SCHEMA branch;
```

```
GO
```

```
CREATE SCHEMA staff;
```

```
GO
```

```
CREATE SCHEMA currency;
```

```
GO
```

```
CREATE SCHEMA account;
```

```
GO
```

```
CREATE SCHEMA device;
```

```
GO
```

```
CREATE SCHEMA card;
```

```
GO
```

```
CREATE SCHEMA atm;
```

```
GO
```

```
CREATE SCHEMA trx;
```

```
GO
```

```
CREATE SCHEMA cheque;
```

```
GO
```

```
CREATE SCHEMA loan;
```

```
GO
```

```
CREATE SCHEMA message;
```

```
GO
```

```
CREATE SCHEMA ledger;
```

```
GO
```

```
/* =====  
CUSTOMER  
===== */
```

```
CREATE TABLE customer.customer
```

```
(
```

```
customer_id INT IDENTITY(1,1) PRIMARY KEY,
```

```
customer_type VARCHAR(20) NOT NULL
```

```
CHECK (customer_type IN ('individual', 'organization')),
```

```
phone VARCHAR(20) NOT NULL UNIQUE,
```

```
email VARCHAR(100) NOT NULL UNIQUE,
```

```
password_hash VARBINARY(256) NOT NULL,  
status VARCHAR(20) NOT NULL DEFAULT 'active'  
CHECK (status IN ('active', 'suspended', 'closed', 'dormant')),  
created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),  
updated_at DATETIME2 NOT NULL DEFAULT SYSDATETIME()  
);  
GO
```

```
CREATE TABLE customer.individual_customer  
(  
customer_id INT PRIMARY KEY,  
first_name VARCHAR(60) NOT NULL,  
last_name VARCHAR(60) NOT NULL,  
birth_date DATE NOT NULL,  
ssn VARBINARY(256) NOT NULL UNIQUE,  
occupation VARCHAR(50) NULL,  
address VARCHAR(256) NULL,  
CONSTRAINT FK_individual_customer_customer  
FOREIGN KEY (customer_id)  
REFERENCES customer.customer(customer_id)  
ON DELETE NO ACTION,  
CONSTRAINT CK_individual_customer_birth_date  
CHECK (birth_date <= CAST(GETDATE() AS DATE)),  
CONSTRAINT CK_individual_customer_age  
CHECK (DATEDIFF(YEAR, birth_date, CAST(GETDATE() AS DATE)) >= 18)  
);  
GO
```

```
CREATE TABLE customer.organization_customer  
(  
customer_id INT PRIMARY KEY,  
organization_name VARCHAR(100) NOT NULL,  
registration_number VARBINARY(256) NOT NULL UNIQUE,  
founded_date DATE NULL,  
industry VARCHAR(50) NULL,  
headquarters_address VARCHAR(256) NULL,  
contact_person_id INT NULL,  
ceo_ssn VARBINARY(256) NULL,  
CONSTRAINT FK_organization_customer_customer  
FOREIGN KEY (customer_id)  
REFERENCES customer.customer(customer_id)  
ON DELETE NO ACTION,  
CONSTRAINT FK_organization_customer_contact_person  
FOREIGN KEY (contact_person_id)  
REFERENCES customer.individual_customer(customer_id)  
ON DELETE SET NULL,  
CONSTRAINT CK_organization_customer_founded_date  
CHECK (founded_date IS NULL OR founded_date <= CAST(GETDATE() AS DATE))  
);  
GO
```

```

/* =====
BRANCH
===== */
CREATE TABLE branch.branch
(
branch_id INT IDENTITY(1,1) PRIMARY KEY,
branch_code VARCHAR(20) NOT NULL UNIQUE,
branch_name VARCHAR(100) NOT NULL,
city VARCHAR(50) NOT NULL,
address VARCHAR(256) NOT NULL,
establish_date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (status IN ('active', 'closed', 'renovating'))
);
GO

/* =====
STAFF
===== */
CREATE TABLE staff.staff
(
staff_id INT IDENTITY(1,1) PRIMARY KEY,
first_name VARCHAR(60) NOT NULL,
last_name VARCHAR(60) NOT NULL,
ssn VARBINARY(256) NOT NULL UNIQUE,
email VARCHAR(100) NOT NULL UNIQUE,
phone VARCHAR(20) NULL UNIQUE,
password_hash VARBINARY(256) NOT NULL,
role VARCHAR(20) NOT NULL
CHECK (role IN ('employee', 'manager', 'admin', 'teller', 'auditor')),
address VARCHAR(256) NULL,
hire_date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
termination_date DATE NULL,
branch_id INT NULL,
created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
updated_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (status IN ('active', 'inactive', 'on_leave')),
CONSTRAINT FK_staff_branch
FOREIGN KEY (branch_id)
REFERENCES branch.branch(branch_id)
ON DELETE SET NULL
);
GO

/* =====
CURRENCY
===== */
CREATE TABLE currency.currency

```

```
(
currency_id INT IDENTITY(1,1) PRIMARY KEY,
currency_code CHAR(3) NOT NULL UNIQUE,
currency_name VARCHAR(50) NOT NULL,
currency_symbol VARCHAR(5) NOT NULL,
is_foreign BIT NOT NULL DEFAULT 0
);
GO
```

```
/* =====
ACCOUNT
Ownership is normalized through account_owner only.
===== */
```

```
CREATE TABLE account.account
(
account_id INT IDENTITY(1,1) PRIMARY KEY,
account_number CHAR(16) NOT NULL UNIQUE,
currency_id INT NOT NULL,
account_type VARCHAR(20) NOT NULL
CHECK (account_type IN ('saving', 'current', 'business', 'loan_facility', 'investment')),
balance DECIMAL(18,2) NOT NULL DEFAULT 0
CHECK (balance >= 0),
account_status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (account_status IN ('active', 'blocked', 'closed', 'dormant', 'pending_closure')),
created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
updated_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
closed_at DATETIME2 NULL,
CONSTRAINT FK_account_currency
FOREIGN KEY (currency_id)
REFERENCES currency.currency(currency_id),
CONSTRAINT CK_account_closed_at
CHECK (
(account_status = 'closed' AND closed_at IS NOT NULL)
OR (account_status <> 'closed' AND closed_at IS NULL)
)
);
GO
```

```
CREATE TABLE account.account_owner
(
account_id INT NOT NULL,
customer_id INT NOT NULL,
ownership_type VARCHAR(20) NOT NULL DEFAULT 'primary'
CHECK (ownership_type IN ('primary', 'joint', 'authorized_signer', 'beneficiary')),
assigned_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (status IN ('active', 'inactive')),
PRIMARY KEY (account_id, customer_id),
CONSTRAINT FK_account_owner_account
FOREIGN KEY (account_id)
```

```
REFERENCES account.account(account_id)
ON DELETE NO ACTION,
CONSTRAINT FK_account_owner_customer
FOREIGN KEY (customer_id)
REFERENCES customer.customer(customer_id)
ON DELETE NO ACTION
);
GO
```

```
/* =====
DEVICE
===== */
CREATE TABLE device.device
(
device_id INT IDENTITY(1,1) PRIMARY KEY,
device_type VARCHAR(20) NOT NULL
CHECK (device_type IN ('ATM', 'POS', 'MOBILE_APP', 'WEB_BROWSER', 'TELLER_TERMINAL')),
customer_id INT NULL,
serial_number VARCHAR(100) NULL UNIQUE,
device_identifier VARCHAR(100) NULL,
registered_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
last_seen_at DATETIME2 NULL,
status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (status IN ('active', 'deactivated', 'maintenance')),
CONSTRAINT FK_device_customer
FOREIGN KEY(customer_id) REFERENCES customer.customer(customer_id) ON DELETE NO ACTION
);
GO
```

```
/* =====
CARD
===== */
CREATE TABLE card.card
(
card_id INT IDENTITY(1,1) PRIMARY KEY,
card_number CHAR(16) NOT NULL UNIQUE,
account_id INT NOT NULL UNIQUE,
expire_date DATE NOT NULL,
cvv2 VARBINARY(256) NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (status IN ('active', 'blocked', 'expired', 'cancelled')),
issued_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
CONSTRAINT FK_card_account
FOREIGN KEY (account_id)
REFERENCES account.account(account_id)
ON DELETE NO ACTION,
CONSTRAINT CK_card_expire_date
CHECK (expire_date > CAST(GETDATE() AS DATE))
);
GO
```

```
/* =====  
ATM  
===== */
```

```
CREATE TABLE atm.atm  
(  
  atm_id INT IDENTITY(1,1) PRIMARY KEY,  
  branch_id INT NULL,  
  city VARCHAR(50) NOT NULL,  
  address VARCHAR(256) NOT NULL,  
  status VARCHAR(20) NOT NULL DEFAULT 'active'  
  CHECK (status IN ('active', 'inactive', 'maintenance')),  
  establish_date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),  
  CONSTRAINT FK_atm_branch  
  FOREIGN KEY (branch_id)  
  REFERENCES branch.branch(branch_id)  
  ON DELETE SET NULL,  
);  
GO
```

```
CREATE TABLE atm.atm_cash  
(  
  atm_id INT NOT NULL,  
  currency_id INT NOT NULL,  
  amount DECIMAL(18,2) NOT NULL DEFAULT 0  
  CHECK (amount >= 0),  
  PRIMARY KEY (atm_id, currency_id),  
  CONSTRAINT FK_atm_cash_atm  
  FOREIGN KEY (atm_id)  
  REFERENCES atm.atm(atm_id)  
  ON DELETE NO ACTION,  
  CONSTRAINT FK_atm_cash_currency  
  FOREIGN KEY (currency_id)  
  REFERENCES currency.currency(currency_id)  
);  
GO
```

```
/* =====  
TRANSACTIONS  
===== */
```

```
CREATE TABLE trx.transactions  
(  
  transaction_id BIGINT IDENTITY(1,1) PRIMARY KEY,  
  reference_code VARCHAR(50) NOT NULL UNIQUE,  
  source_account_id INT NULL,  
  target_account_id INT NULL,  
  source_device_id INT NULL,  
  transaction_type VARCHAR(20) NOT NULL  
  CHECK (transaction_type IN ('transfer', 'withdraw', 'deposit', 'purchase', 'bill_payment')),
```

```

amount DECIMAL(18,2) NOT NULL
CHECK (amount > 0),
transaction_status VARCHAR(20) NOT NULL DEFAULT 'pending'
CHECK (transaction_status IN ('pending', 'successful', 'failed', 'reversed')),
description VARCHAR(256) NULL,
issued_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
completed_at DATETIME2 NULL,
CONSTRAINT FK_transactions_source_account
FOREIGN KEY (source_account_id)
REFERENCES account.account(account_id)
ON DELETE NO ACTION,
CONSTRAINT FK_transactions_target_account
FOREIGN KEY (target_account_id)
REFERENCES account.account(account_id)
ON DELETE NO ACTION,
CONSTRAINT FK_transactions_device
FOREIGN KEY (source_device_id)
REFERENCES device.device(device_id)
ON DELETE NO ACTION,
CONSTRAINT CK_transactions_self_transfer
CHECK (source_account_id IS NULL
OR target_account_id IS NULL
OR source_account_id <> target_account_id),
CONSTRAINT CK_transactions_type_structure
CHECK
(
(transaction_type = 'transfer'
AND source_account_id IS NOT NULL
AND target_account_id IS NOT NULL)
OR
(transaction_type = 'withdraw'
AND source_account_id IS NOT NULL
AND target_account_id IS NULL)
OR
(transaction_type = 'deposit'
AND source_account_id IS NULL
AND target_account_id IS NOT NULL)
OR
(transaction_type = 'purchase'
AND source_account_id IS NOT NULL
AND target_account_id IS NULL)
OR
(transaction_type = 'bill_payment'
AND source_account_id IS NOT NULL
AND target_account_id IS NOT NULL)
)
);
GO

```

```

/* =====

```

CHEQUE / CHECKBOOK

```
===== */
CREATE TABLE cheque.checkbook
(
  checkbook_id INT IDENTITY(1,1) PRIMARY KEY,
  account_id INT NOT NULL,
  branch_id INT NOT NULL,
  issue_date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
  number_of_checks INT NOT NULL
  CHECK (number_of_checks > 0),
  status VARCHAR(20) NOT NULL DEFAULT 'active'
  CHECK (status IN ('active', 'issued', 'cancelled', 'closed')),
  CONSTRAINT FK_checkbook_account
  FOREIGN KEY (account_id)
  REFERENCES account.account(account_id)
  ON DELETE NO ACTION,
  CONSTRAINT FK_checkbook_branch
  FOREIGN KEY (branch_id)
  REFERENCES branch.branch(branch_id)
  ON DELETE NO ACTION
);
GO

CREATE TABLE cheque.check_paper
(
  check_id INT IDENTITY(1,1) PRIMARY KEY,
  check_number VARCHAR(30) NOT NULL UNIQUE,
  checkbook_id INT NOT NULL,
  receiver_account_id INT NOT NULL,
  amount DECIMAL(18,2) NOT NULL
  CHECK (amount > 0),
  issued_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
  expire_date DATE NOT NULL,
  cleared_date DATE NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'issued'
  CHECK (status IN ('issued', 'cashed', 'bounced', 'cancelled')),
  CONSTRAINT FK_check_paper_checkbook
  FOREIGN KEY (checkbook_id)
  REFERENCES cheque.checkbook(checkbook_id)
  ON DELETE NO ACTION,
  CONSTRAINT FK_check_paper_receiver_account
  FOREIGN KEY (receiver_account_id)
  REFERENCES account.account(account_id)
  ON DELETE NO ACTION,
  CONSTRAINT CK_check_paper_expire_date
  CHECK (expire_date >= CAST(issued_at AS DATE))
);
GO

/* =====
```


LOAN / FACILITY

```
===== */
CREATE TABLE loan.loan
(
loan_id INT IDENTITY(1,1) PRIMARY KEY,
account_id INT NOT NULL,
guarantor_customer_id INT NOT NULL,
amount DECIMAL(18,2) NOT NULL
CHECK (amount > 0),
interest_rate DECIMAL(5,2) NOT NULL
CHECK (interest_rate >= 0 AND interest_rate <= 100),
loan_term_months INT NOT NULL
CHECK (loan_term_months > 0),
repayment_status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (repayment_status IN ('active', 'completed', 'defaulted')),
issue_date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
CONSTRAINT FK_loan_account
FOREIGN KEY (account_id)
REFERENCES account.account(account_id)
ON DELETE NO ACTION,
CONSTRAINT FK_loan_guarantor_customer
FOREIGN KEY (guarantor_customer_id)
REFERENCES customer.customer(customer_id)
ON DELETE NO ACTION
);
GO
```

```
/* =====
MESSAGE
===== */
CREATE TABLE message.message
(
message_id INT IDENTITY(1,1) PRIMARY KEY,
customer_id INT NOT NULL,
subject VARCHAR(150) NOT NULL,
body NVARCHAR(MAX) NOT NULL,
message_status VARCHAR(20) NOT NULL DEFAULT 'unread'
CHECK (message_status IN ('unread', 'read', 'archived', 'deleted')),
created_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
read_at DATETIME2 NULL,
CONSTRAINT FK_message_customer
FOREIGN KEY (customer_id)
REFERENCES customer.customer(customer_id)
ON DELETE NO ACTION
);
GO
```

```
/* =====
HISTORICAL LEDGER / BALANCE TRACKING
===== */
```

```
CREATE TABLE ledger.account_balance_history
(
  history_id BIGINT IDENTITY(1,1) PRIMARY KEY,
  account_id INT NOT NULL,
  old_balance DECIMAL(18,2) NOT NULL,
  new_balance DECIMAL(18,2) NOT NULL,
  changed_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
  changed_by_transaction_id BIGINT NULL,
  changed_by_user_id INT NULL,
  CONSTRAINT FK_balance_history_account
  FOREIGN KEY (account_id)
  REFERENCES account.account(account_id)
  ON DELETE NO ACTION
);
GO
```

```
CREATE TABLE ledger.transaction_status_history
(
  history_id BIGINT IDENTITY(1,1) PRIMARY KEY,
  transaction_id BIGINT NOT NULL,
  old_status VARCHAR(20) NOT NULL,
  new_status VARCHAR(20) NOT NULL,
  changed_at DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
  changed_by_user_id INT NULL,
  note VARCHAR(256) NULL,
  CONSTRAINT FK_transaction_status_history_transaction
  FOREIGN KEY (transaction_id)
  REFERENCES trx.transactions(transaction_id)
  ON DELETE NO ACTION
);
GO
```

7) اجزای برنامه‌نویسی پایگاه داده

7-1) Stored Procedures

رویه‌های ذخیره‌شده برای انجام عملیات‌های اصلی سامانه استفاده شده‌اند.

این عملیات‌ها شامل:

- ورود به سیستم
- ایجاد مشتری حقیقی و حقوقی
- حذف مشتری

- دریافت اطلاعات مشتری
- تغییر وضعیت مشتری
- ایجاد حساب
- حذف مالک حساب
- ایجاد شعبه
- ایجاد کارمند
- تغییر شعبه کارمند
- ایجاد کارت
- ایجاد ATM
- شارژ ATM
- ثبت دستگاه
- صدور چک بوک
- صدور چک
- ایجاد وام

مزایای استفاده از Stored Procedure

- یکپارچگی منطق
- کاهش تکرار کد
- اجرای امن تر عملیات
- امکان استفاده از تراکنش
- کنترل خطا

Stored procedures

-- login user with email and password

CREATE procedure usp_LoginWithEmailPass

@Email varchar(100),

@Password varchar(100)

as

begin

DECLARE @Login INT = 1;

SELECT @Login = 0

FROM customer.customer

WHERE email = @Email AND password_hash = HASHBYTES('SHA2_256',@Password)

```
SELECT @Login = 0
FROM staff.staff
WHERE email = @Email AND password_hash = HASHBYTES('SHA2_256',@Password)
```

```
-- if login is 0 the login is successfull
```

```
RETURN @Login;
END;
GO
```

```
-- create an individual customer
```

```
CREATE PROCEDURE usp_CreateIndividualCustomer
```

```
@CustomerType VARCHAR(20) = 'individual',
@Phone VARCHAR(20),
@Email VARCHAR(100),
@Password VARCHAR(100),
@Status VARCHAR(20) = 'active',
@FirstName VARCHAR(60),
@LastName VARCHAR(60),
@BirthDate DATE,
@SSN VARCHAR(15),
@Occupation VARCHAR(50),
@Address VARCHAR(256)
```

```
AS
```

```
BEGIN
```

```
BEGIN TRY;
```

```
BEGIN TRANSACTION;
```

```
INSERT into customer.customer
```

```
(customer_type,phone,email,password_hash,[status])
```

```
VALUES
```

```
(@CustomerType, @Phone, @Email, HASHBYTES('SHA2_256', @Password), @Status)
```

```
DECLARE @CustId INT = 0;
```

```
SELECT
```

```
@CustId = customer_id
```

```
from customer.customer
```

```
WHERE email = @Email AND password_hash = HASHBYTES('SHA2_256',@Password)
```

```
IF @CustId = 0
```

```
BEGIN
```

```
ROLLBACK TRANSACTION;
```

```
RETURN 1;
```

```
END
```

```
INSERT into customer.individual_customer
```

```
(customer_id,first_name,last_name,birth_date,ssn,occupation,address)
```

```
VALUES
```

```
(@CustId, @FirstName, @LastName, @BirthDate, HASHBYTES('SHA2_256', @SSN), @Occupation, @Address)
```

```
COMMIT TRANSACTION;
```

```

-- success
RETURN 0;
END TRY
BEGIN CATCH
IF @@TRANCOUNT>0
ROLLBACK TRANSACTION;
THROW;
end CATCH
END;
GO

-- get and individual customer with cust id
CREATE PROCEDURE usp_GetIndCustomer
@CustId INT
AS
BEGIN

SELECT *
FROM customer.customer c
JOIN customer.individual_customer cc on cc.customer_id = c.customer_id
WHERE c.customer_id = @CustId AND c.customer_type = 'individual'

END;
GO

-- delete an ind customer
CREATE PROCEDURE usp_DelIndCustomer
@CustId INT
AS
BEGIN

IF EXISTS(
SELECT 1
FROM customer.customer c
JOIN customer.individual_customer cc on cc.customer_id = c.customer_id
WHERE c.customer_id = @CustId AND c.customer_type = 'individual'
)
BEGIN

BEGIN TRANSACTION;
DELETE FROM customer.customer
WHERE customer_id = @CustId

DELETE from customer.individual_customer
where customer_id = @CustId

COMMIT TRANSACTION;

END
END;

```

GO

-- delete an org customer

CREATE PROCEDURE usp_DelOrgCustomer

@CustId INT

AS

BEGIN

IF EXISTS(

SELECT 1

FROM customer.customer c

JOIN customer.organization_customer cc on cc.customer_id = c.customer_id

WHERE c.customer_id = @CustId AND c.customer_type = 'organization'

)

BEGIN

BEGIN TRANSACTION;

DELETE FROM customer.customer

WHERE customer_id = @CustId

DELETE from customer.organization_customer

where customer_id = @CustId

COMMIT TRANSACTION;

END

END;

GO

-- create an organization customer

CREATE PROCEDURE usp_CreateOrgCustomer

@CustomerType VARCHAR(20) = 'organization',

@Phone VARCHAR(20),

@Email VARCHAR(100),

@Password VARCHAR(100),

@Status VARCHAR(20) = 'active',

@OrgName VARCHAR(100),

@RegNum VARCHAR(100),

@FoundDate DATE = NULL,

@Industry VARCHAR(50) = "",

@HqAddr VARCHAR(256) = "",

@ContPersonId INT = NULL,

@CeoSsn VARCHAR(15)

AS

BEGIN

BEGIN TRY

BEGIN TRANSACTION;

```

INSERT into customer.customer
(customer_type,phone,email,password_hash,[status])
VALUES
(@CustomerType, @Phone, @Email, HASHBYTES('SHA2_256', @Password), @Status)

DECLARE @CustId INT = 0;

SELECT @CustId = customer_id
from customer.customer
WHERE email = @Email AND password_hash = HASHBYTES('SHA2_256',@Password)

IF @CustId = 0
BEGIN
ROLLBACK TRANSACTION;
RETURN 1;
END

INSERT into customer.organization_customer
(customer_id,organization_name,registration_number,founded_date,industry,headquarters_address)
VALUES(
@CustId, @OrgName, HASHBYTES('SHA2_256', @RegNum), @FoundDate, @Industry, @HqAddr
)

COMMIT TRANSACTION;
-- success
RETURN 0;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0
ROLLBACK TRANSACTION;
THROW;
END CATCH

END;
GO

-- get customer generic customer info
CREATE PROCEDURE usp_GetCust
@CustId INT
AS
BEGIN
SELECT
c.customer_type,
c.email,
c.phone,
c.created_at,
c.[status],

```

```
c.updated_at
FROM customer.customer c
WHERE c.customer_id = @CustId
END;
GO
```

-- change customer status

```
CREATE PROCEDURE usp_PatchCustStatus
@CustId INT,
@Status VARCHAR(20) = 'suspend'
AS
BEGIN
```

```
UPDATE customer.customer
SET
[status] = @Status
WHERE customer_id = @CustId
```

```
END;
GO
```

-- get all customer accounts

```
CREATE PROCEDURE usp_GetCustAcc
@CustId INT,
@Status VARCHAR(20) = NULL,
@Type VARCHAR(20) = NULL
AS
BEGIN
```

```
SELECT
*
FROM account.account_owner a
JOIN account.account ac on ac.account_id = a.account_id
WHERE a.customer_id = @CustId AND (@Status is null OR a.[status] = @Status) AND (@Type is null
or a.ownership_type = @Type)
END;
GO
```

-- create new account for a customer

```
CREATE PROCEDURE usp_CreatAcc
@CustId INT,
@AccNumber CHAR(16),
@CurrencyId INT,
@AccType VARCHAR(20),
@Balance DECIMAL(18,2) = 0,
@AccStatus VARCHAR(20)
AS
BEGIN
BEGIN TRY
```



```

BEGIN TRANSACTION;

-- create account first
INSERT into account.account
(account_number,currency_id,account_type,balance,account_status)
VALUES(
@AccNumber, @CurrencyId, @AccType, @Balance, @AccStatus
)

DECLARE @AccId INT = 0;
SELECT
@AccId = account_id
FROM account.account
WHERE account_number = @AccNumber

IF @AccId = 0
BEGIN
RAISERROR('Account did not created', 16, 1);
END

-- create account owner ship and set it for customer
INSERT into account.account_owner
(
account_id,customer_id,ownership_type,[status]
)
VALUES(
@AccId, @CustId, 'primary', 'active'
)

COMMIT TRANSACTION;

-- success
RETURN 0;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0
ROLLBACK TRANSACTION;
THROW;
END CATCH

END;
GO

-- get balance of the account id
CREATE PROCEDURE usp_GetAccBal
@AccId INT
AS
BEGIN
SELECT account_number, balance

```

```

FROM account.account
WHERE account_id = @AccId
END
GO

-- remove and owener from an account
CREATE PROCEDURE usp_RemOwnAcc
@CustId INT,
@AccId INT
AS
BEGIN
DECLARE @Type VARCHAR(20);
SELECT @Type = ownership_type
FROM account.account_owner
WHERE account_id = @AccId AND customer_id = @CustId

IF @Type = 'primary'
BEGIN
RAISERROR('primary ownership cant be removed',16,1);
END

DELETE account.account_owner WHERE account_id = @AccId AND customer_id = @CustId

END
GO

```

•

7-2) Functions

توابع تعریف شده در سیستم بیشتر برای گزارش گیری و تحلیل هستند.

این توابع خروجی جدولی ارائه می دهند و برای پرس و جوهای مدیریتی استفاده می شوند.

نمونه کاربردهای آنها

• خلاصه کارکنان شعبه

• تحلیل وام و چک

• نمای کلی تعامل مشتری

این توابع کمک می کنند اطلاعات پراکنده در قالب های قابل فهم و تجمیع شده ارائه شوند.

Functions

```

-- BRANCH & STAFF
CREATE OR ALTER FUNCTION branch.fn_GetBranchStaffSummary()
RETURNS TABLE

```

```

AS
RETURN
(
SELECT
b.branch_id,
b.branch_code,
b.branch_name,
b.city,
COUNT(s.staff_id) AS total_staff,
SUM(CASE WHEN s.status = 'active' THEN 1 ELSE 0 END) AS active_staff,
SUM(CASE WHEN s.role = 'manager' THEN 1 ELSE 0 END) AS manager_count,
MAX(s.hire_date) AS latest_hire_date
FROM branch.branch b
LEFT JOIN staff.staff s ON b.branch_id = s.branch_id
GROUP BY b.branch_id, b.branch_code, b.branch_name, b.city
);
GO

```

-- DEBT, LIABILITIES, AND COMMITTED FUND ANALYTICS

```

CREATE OR ALTER FUNCTION loan.fn_GetLiabilityAndPaperSummary()
RETURNS TABLE
AS
RETURN
(
SELECT
a.account_number,
a.account_type,
-- Loan aggregation
COUNT(DISTINCT l.loan_id) AS total_loans_issued,
ISNULL(SUM(l.amount), 0) AS aggregate_loan_principal,
ISNULL(AVG(l.interest_rate), 0) AS average_loan_rate,
-- Cheque clearance processing
COUNT(DISTINCT cb.checkbook_id) AS checkbooks_allocated,
COUNT(DISTINCT cp.check_id) AS total_checks_written,
ISNULL(SUM(CASE WHEN cp.status = 'bounced' THEN cp.amount ELSE 0 END), 0) AS
total_bounced_value
FROM account.account a
LEFT JOIN loan.loan l ON a.account_id = l.account_id AND l.repayment_status = 'active'
LEFT JOIN cheque.checkbook cb ON a.account_id = cb.account_id
LEFT JOIN cheque.check_paper cp ON cp.checkbook_id = cb.checkbook_id
GROUP BY a.account_number, a.account_type
);
GO

```

-- CUSTOMER ENGAGEMENT

```

CREATE OR ALTER FUNCTION customer.fn_GetCustomerEngagementOverview()
RETURNS TABLE
AS
RETURN
(

```

```

SELECT
c.customer_id,
c.customer_type,
c.status AS registration_status,
-- Checks configuration differences between Individuals and Organizations
COALESCE(ic.first_name + ' ' + ic.last_name, oc.organization_name) AS entity_display_name,
-- Account attachment count
COUNT(DISTINCT ao.account_id) AS active_accounts_owned,
-- Cards provisioned
COUNT(DISTINCT card.card_id) AS plastic_cards_issued,
-- Customer interactions
COUNT(DISTINCT m.message_id) AS secure_messages_sent
FROM customer.customer c
LEFT JOIN customer.individual_customer ic ON c.customer_id = ic.customer_id
LEFT JOIN customer.organization_customer oc ON c.customer_id = oc.customer_id
LEFT JOIN account.account_owner ao ON c.customer_id = ao.customer_id
LEFT JOIN card.card card ON ao.account_id = card.account_id
LEFT JOIN message.message m ON c.customer_id = m.customer_id
GROUP BY c.customer_id, c.customer_type, c.status, ic.first_name, ic.last_name,
oc.organization_name
);
GO

```

7-3) Triggers

تریگرها برای ثبت خودکار رویدادهای حساس استفاده شده‌اند.

تریگرهای مهم

- ثبت تاریخچه تغییر موجودی حساب
- ثبت تاریخچه تغییر وضعیت تراکنش

مزیت

- بدون نیاز به دخالت دستی، تاریخچه‌ها ثبت می‌شوند
- احتمال فراموشی یا خطای انسانی کاهش می‌یابد

Triggers

```

CREATE TRIGGER account.trg_account_balance_history
ON account.account
AFTER UPDATE
AS
BEGIN

```

```
SET NOCOUNT ON;
```

```
INSERT INTO ledger.account_balance_history
(
    account_id,
    old_balance,
    new_balance,
    changed_at,
    changed_by_transaction_id,
    changed_by_user_id
)
SELECT
    d.account_id,
    d.balance,
    i.balance,
    SYSDATETIME(),
    NULL,
    NULL
FROM inserted i
INNER JOIN deleted d
ON i.account_id = d.account_id
WHERE ISNULL(i.balance, -1) <> ISNULL(d.balance, -1);
END;
GO
```

```
CREATE TRIGGER trx.trg_transaction_status_history
ON trx.transactions
AFTER UPDATE
AS
BEGIN
SET NOCOUNT ON;
```

```
INSERT INTO ledger.transaction_status_history
(
    transaction_id,
    old_status,
    new_status,
    changed_at,
    changed_by_user_id,
    note
)
SELECT
    d.transaction_id,
    d.transaction_status,
    i.transaction_status,
    SYSDATETIME(),
    NULL,
    NULL
FROM inserted i
INNER JOIN deleted d
```

```
ON i.transaction_id = d.transaction_id
WHERE ISNULL(i.transaction_status, '') <> ISNULL(d.transaction_status, '');
END;
GO
```

```
CREATE TRIGGER trx.trg_CheckSourceBalance
ON trx.transactions
INSTEAD OF INSERT
AS
BEGIN
SET NOCOUNT ON;
```

```
-- Check for insufficient balance
```

```
IF EXISTS (
SELECT 1
FROM inserted i
JOIN account.account a WITH (UPDLOCK, HOLDLOCK)
ON a.account_id = i.source_account_id
WHERE i.source_account_id IS NOT NULL
AND i.transaction_type IN ('transfer', 'withdraw', 'purchase', 'bill_payment')
AND a.balance < i.amount
)
BEGIN
RAISERROR ('Insufficient balance in source account.', 16, 1);
ROLLBACK TRANSACTION;
RETURN;
END
```

```
-- If everything is valid, insert the rows
```

```
INSERT INTO trx.transactions
(
reference_code,
source_account_id,
target_account_id,
source_device_id,
transaction_type,
amount,
transaction_status,
description,
issued_at,
completed_at
)
SELECT
reference_code,
source_account_id,
target_account_id,
source_device_id,
transaction_type,
amount,
transaction_status,
```

```
description,  
issued_at,  
completed_at  
FROM inserted;  
END;  
GO
```

7-4) Indexes

ایندکس‌ها برای افزایش سرعت جست‌وجو و گزارش‌گیری استفاده شده‌اند.

روی ستون‌هایی مانند:

- وضعیت مشتری
- شماره ملی/شناسه مشتری
- شعبه کارمند
- ارز حساب
- وضعیت حساب
- حساب مالک
- حساب کارت
- شعبه ATM
- حساب مبدا/مقصد تراکنش
- وضعیت تراکنش
- وام و ضامن
- پیام مشتری
- تاریخچه حساب

ایندکس تعریف شده است.

اثر ایندکس

- افزایش سرعت خواندن داده
- بهبود عملکرد joinها
- مناسب برای گزارش‌های پرتکرار

Indexes

```
-- customer.customer
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_customer_status' AND object_id = OBJECT_ID('customer.customer'))
CREATE INDEX IX_customer_status ON customer.customer([status]);
GO
```

```
-- customer.individual_customer
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_individual_customer_ssn' AND object_id =
OBJECT_ID('customer.individual_customer'))
CREATE INDEX IX_individual_customer_ssn ON customer.individual_customer(ssn);
GO
```

```
-- customer.organization_customer
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_organization_customer_contact_person' AND object_id =
OBJECT_ID('customer.organization_customer'))
CREATE INDEX IX_organization_customer_contact_person ON
customer.organization_customer(contact_person_id);
GO
```

```
-- staff.staff
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_staff_branch' AND object_id = OBJECT_ID('staff.staff'))
CREATE INDEX IX_staff_branch ON staff.staff(branch_id);
GO
```

```
-- account.account
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_account_currency' AND object_id = OBJECT_ID('account.account'))
CREATE INDEX IX_account_currency ON account.account(currency_id);
GO
```

```
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_account_status' AND object_id = OBJECT_ID('account.account'))
CREATE INDEX IX_account_status ON account.account(account_status);
GO
```

```
-- account.account_owner
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_account_owner_customer' AND object_id =
OBJECT_ID('account.account_owner'))
```



```

CREATE INDEX IX_account_owner_customer ON account.account_owner(customer_id);
GO

-- card.card
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_card_account' AND object_id = OBJECT_ID('card.card'))
CREATE INDEX IX_card_account ON card.card(account_id);
GO

-- atm.atm
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_atm_branch' AND object_id = OBJECT_ID('atm.atm'))
CREATE INDEX IX_atm_branch ON atm.atm(branch_id);
GO

-- atm.atm_cash
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_atm_cash_currency' AND object_id = OBJECT_ID('atm.atm_cash'))
CREATE INDEX IX_atm_cash_currency ON atm.atm_cash(currency_id);
GO

-- trx.transactions
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_transactions_source' AND object_id = OBJECT_ID('trx.transactions'))
CREATE INDEX IX_transactions_source ON trx.transactions(source_account_id);
GO

IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_transactions_target' AND object_id = OBJECT_ID('trx.transactions'))
CREATE INDEX IX_transactions_target ON trx.transactions(target_account_id);
GO

IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_transactions_device' AND object_id = OBJECT_ID('trx.transactions'))
CREATE INDEX IX_transactions_device ON trx.transactions(source_device_id);
GO

IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_transactions_status' AND object_id = OBJECT_ID('trx.transactions'))
CREATE INDEX IX_transactions_status ON trx.transactions(transaction_status);
GO

-- cheque.checkbook

```

```
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_checkbook_account' AND object_id = OBJECT_ID('cheque.checkbook'))
CREATE INDEX IX_checkbook_account ON cheque.checkbook(account_id);
GO
```

```
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_checkbook_branch' AND object_id = OBJECT_ID('cheque.checkbook'))
CREATE INDEX IX_checkbook_branch ON cheque.checkbook(branch_id);
GO
```

```
-- cheque.check_paper
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_check_paper_checkbook' AND object_id = OBJECT_ID('cheque.check_paper'))
CREATE INDEX IX_check_paper_checkbook ON cheque.check_paper(checkbook_id);
GO
```

```
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_check_paper_payer_account' AND object_id =
OBJECT_ID('cheque.check_paper'))
CREATE INDEX IX_check_paper_payer_account ON cheque.check_paper(receiver_account_id);
GO
```

```
-- loan.loan
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_loan_account' AND object_id = OBJECT_ID('loan.loan'))
CREATE INDEX IX_loan_account ON loan.loan(account_id);
GO
```

```
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_loan_guarantor_customer' AND object_id = OBJECT_ID('loan.loan'))
CREATE INDEX IX_loan_guarantor_customer ON loan.loan(guarantor_customer_id);
GO
```

```
-- message.message
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_message_customer' AND object_id = OBJECT_ID('message.message'))
CREATE INDEX IX_message_customer ON message.message(customer_id);
GO
```

```
-- ledger.ledger_account
IF NOT EXISTS (SELECT 1
FROM sys.indexes
WHERE name = 'IX_ledger_parent' AND object_id = OBJECT_ID('ledger.account_balance_history'))
```

```
CREATE INDEX IX_ledger_parent ON ledger.account_balance_history(account_id);  
GO
```

7-5) RBAC

در این سیستم کنترل دسترسی مبتنی بر نقش پیاده‌سازی شده است.

نقش‌ها

- کاربر اپلیکیشن
- کارمند باجه
- حسابرس
- مدیر شعبه

الگوی دسترسی

- کاربر اپلیکیشن: دسترسی عملیاتی محدود
- کارمند باجه: مشاهده اطلاعات و ثبت تراکنش
- حسابرس: فقط خواندن
- مدیر: نقش مدیریتی

اهمیت

این بخش امنیت سیستم را بسیار بالا می‌برد و از دسترسی غیرمجاز جلوگیری می‌کند.

Rbac

```
/* Create Roles */
```

```
CREATE ROLE Admin;  
-- The role for the application's service account  
CREATE ROLE Teller;  
-- For branch staff  
CREATE ROLE Customer;  
-- For internal compliance  
CREATE ROLE Manager;  
-- For branch management
```

```
/* 1. Permissions for the Application (The Service Account) */  
-- The application needs to read/write transactions, accounts, and customers  
GRANT SELECT, INSERT, UPDATE ON SCHEMA::account TO Admin;
```

```
GRANT SELECT, INSERT, UPDATE ON SCHEMA::trx TO Admin;  
GRANT SELECT, INSERT, UPDATE ON SCHEMA::customer TO Admin;  
GRANT SELECT, INSERT, UPDATE ON SCHEMA::device TO Admin;  
GRANT SELECT, INSERT ON SCHEMA::ledger TO Admin;
```

```
GRANT SELECT, INSERT, UPDATE ON SCHEMA::account TO Manager;  
GRANT SELECT, INSERT, UPDATE ON SCHEMA::trx TO Manager;  
GRANT SELECT, INSERT, UPDATE ON SCHEMA::customer TO Manager;  
GRANT SELECT, INSERT, UPDATE ON SCHEMA::device TO Manager;  
GRANT SELECT, INSERT ON SCHEMA::ledger TO Manager;
```

/ 2. Permissions for Tellers */*

```
GRANT SELECT ON SCHEMA::account TO Teller;  
GRANT SELECT, INSERT ON SCHEMA::trx TO Teller;  
GRANT SELECT ON SCHEMA::customer TO Teller;
```

/ 3. Permissions for Auditors (READ ONLY) */*

```
GRANT SELECT ON SCHEMA::account TO Customer;  
GRANT SELECT ON SCHEMA::customer TO Customer;  
GRANT SELECT ON SCHEMA::trx TO Customer;  
GRANT SELECT ON SCHEMA::ledger TO Customer;  
GRANT SELECT ON SCHEMA::loan TO Customer;
```

/ Revoke access to sensitive logic where necessary */*

```
REVOKE DELETE ON SCHEMA::account TO Admin;  
REVOKE DELETE ON SCHEMA::trx TO Admin;
```

7-6) Encryption

برای حفاظت از اطلاعات حساس، از دو رویکرد استفاده شده است:

(1) هاش کردن داده‌های حساس

- رمز عبور
- SSN
- شماره ثبت
- CVV

(2) رمزنگاری سطح پایگاه داده

در سطح دیتابیس نیز encryption فعال شده تا داده‌ها در حالت ذخیره‌شده امن‌تر باشند.

اهمیت

این موضوع در یک سیستم بانکی ضروری است، چون حجم زیادی از اطلاعات حساس نگهداری می‌شود.

Encryption

Step 1: Create a Master Key --

```
USE master;
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Strong_Password_Here!';
```

-- Step 2: Create a Certificate

```
CREATE CERTIFICATE TDE_Cert WITH SUBJECT = 'BankSystemV1_Cert';
```

-- Step 3: Create the Database Encryption Key

```
USE BankSystemV1;
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE TDE_Cert;
```

-- Step 4: Enable Encryption

```
ALTER DATABASE BankSystemV1 SET ENCRYPTION ON;
GO
```

Queries

```
CREATE VIEW customer_account
as
SELECT c.customer_id, ac.*
FROM customer.customer c
JOIN account.account_owner a on c.customer_id = a.customer_id
JOIN account.account ac on ac.account_id = a.account_id
GO
```

-- customers with their owing accounts

```
SELECT c.*, ac.*
FROM customer.customer c
JOIN account.account_owner a on c.customer_id = a.customer_id
JOIN account.account ac on ac.account_id = a.account_id
```

-- customers with active status

```
SELECT
*
FROM customer.customer c
WHERE c.[status] = 'active'
```

-- find active card for a customer

```
SELECT
c.*,
cr.*
```

```
from customer_account c
JOIN card.card cr on cr.account_id = c.account_id
WHERE c.customer_id = 1 AND cr.[status] = 'active'
```

-- select all information on specific customer

```
SELECT
*
FROM customer.customer c
JOIN customer.individual_customer ic on c.customer_id = ic.customer_id
JOIN customer.organization_customer oc on c.customer_id = oc.customer_id
```

-- select customer with phone and email

```
SELECT
*
FROM customer.customer c
WHERE c.email = 'email' AND c.phone = 'phone'
```

-- select customers with all their account ownerships

```
SELECT
*
FROM customer.customer c
JOIN account.account_owner ac on ac.customer_id = c.customer_id
```

-- select org customers with contact person info

```
SELECT
*
FROM customer.organization_customer oc
JOIN customer.individual_customer ic on ic.customer_id = oc.contact_person_id
```

-- select all under 18 customers

```
SELECT
*
FROM customer.individual_customer c
WHERE DATEDIFF(year, CAST(c.birth_date as datetime), CAST(SYSDATETIME() as datetime)) < 18
```

-- select gaurneter of each loan

```
SELECT
*
FROM loan.loan l
JOIN customer.customer c ON l.guarantor_customer_id = c.customer_id
WHERE l.loan_id = 1
```

-- account detilas

```
SELECT
*
FROM account.account a
JOIN currency.currency c on a.currency_id = c.currency_id
```

-- all active accounts

```
SELECT
```

```

*
FROM account.account a
WHERE a.account_status = 'active'

-- all not active accounts
SELECT
*
FROM account.account a
WHERE a.account_status != 'active'

-- get all trasactions history for an account
SELECT
*
FROM trx.transactions t
WHERE t.source_account_id = 1 OR t.target_account_id =1

-- get all transactions for a device
SELECT
*
FROM trx.transactions t
WHERE t.target_account_id = 1

-- all pending transactions
SELECT
*
FROM trx.transactions t
WHERE t.transaction_status = 'pending'

-- total amount of credit and debit on each account
SELECT
t1.account_id,
debit,
credit
FROM
(
SELECT
t.target_account_id as account_id,
SUM(amount) as credit
FROM trx.transactions t
WHERE t.transaction_status = 'successful'
GROUP BY t.target_account_id)t1 JOIN
(
SELECT
t.source_account_id as account_id,
SUM(amount) as debit
FROM trx.transactions t
WHERE t.transaction_status = 'successful'
GROUP BY t.source_account_id)t2 on t1.account_id = t2.account_id

-- select inactive accounts with active cards

```

```

SELECT
*
FROM card.card c
JOIN account.account a on a.account_id = c.account_id
WHERE c.[status] = 'active' and a.account_status != 'active'

-- select all cards from a account
SELECT
*
FROM card.card c
JOIN account.account cc on cc.account_id = c.account_id
WHERE cc.account_id = 1

-- select card expire date within 30 days
SELECT
*
FROM card.card c
WHERE DATEDIFF(day,CAST(SYSDATETIME() as datetime),CAST(c.expire_date as datetime))
BETWEEN 0 and 30

-- select all trx with a card
SELECT
*
FROM trx.transactions t
JOIN card.card c on c.account_id = t.source_account_id or c.account_id = t.target_account_id

-- each loan repayment amount
SELECT
l.loan_id ,
amount*((interest_rate/100) + 1) / loan_term_months as repayment_amount
FROM loan.loan l
WHERE l.repayment_status = 'active'

-- all the cheque from each account
SELECT
a.*,
cp.*
FROM account.account a
JOIN cheque.checkbook c on c.account_id = a.account_id
JOIN cheque.check_paper cp on cp.checkbook_id = c.checkbook_id

```

Transactions

```

USE BankSystemV1;
GO

DECLARE @FromDate date = '2026-01-01';
DECLARE @ToDate date = '2026-01-31';

DECLARE @StartDateTime datetime2 = CAST(@FromDate AS datetime2);

```



```

DECLARE @EndDateTime datetime2 = DATEADD(DAY, 1, CAST(@ToDate AS datetime2));

/* =====
SUCCESSFUL TRANSACTIONS IN PERIOD
===== */
IF OBJECT_ID('tempdb..#tx') IS NOT NULL DROP TABLE #tx;

SELECT
t.transaction_id,
COALESCE(t.completed_at, t.issued_at) AS txn_at,
t.transaction_type,
t.amount
INTO #tx
FROM trx.transactions t
WHERE t.transaction_status = 'successful'
AND COALESCE(t.completed_at, t.issued_at) >= @StartDateTime
AND COALESCE(t.completed_at, t.issued_at) < @EndDateTime;

/* =====
ACCOUNT SNAPSHOT BY CURRENCY
===== */
IF OBJECT_ID('tempdb..#account_agg') IS NOT NULL DROP TABLE #account_agg;

SELECT
a.currency_id,
COUNT(*) AS total_accounts,
SUM(CASE WHEN a.account_status = 'active' THEN 1 ELSE 0 END) AS active_accounts,
SUM(CASE WHEN a.account_status = 'blocked' THEN 1 ELSE 0 END) AS blocked_accounts,
SUM(CASE WHEN a.account_status = 'dormant' THEN 1 ELSE 0 END) AS dormant_accounts,
SUM(CASE WHEN a.account_status = 'closed' THEN 1 ELSE 0 END) AS closed_accounts,
CAST(SUM(a.balance) AS decimal(18,2)) AS total_customer_balances,
CAST(SUM(CASE WHEN a.account_type = 'saving' THEN a.balance ELSE 0 END) AS decimal(18,2)) AS
saving_balance,
CAST(SUM(CASE WHEN a.account_type = 'current' THEN a.balance ELSE 0 END) AS decimal(18,2))
AS current_balance,
CAST(SUM(CASE WHEN a.account_type = 'business' THEN a.balance ELSE 0 END) AS decimal(18,2))
AS business_balance,
CAST(SUM(CASE WHEN a.account_type = 'loan_facility' THEN a.balance ELSE 0 END) AS
decimal(18,2)) AS loan_facility_balance,
CAST(SUM(CASE WHEN a.account_type = 'investment' THEN a.balance ELSE 0 END) AS
decimal(18,2)) AS investment_balance
INTO #account_agg
FROM account.account a
GROUP BY a.currency_id;

/* =====
LOAN SNAPSHOT BY CURRENCY
===== */
IF OBJECT_ID('tempdb..#loan_agg') IS NOT NULL DROP TABLE #loan_agg;

```

```

SELECT
a.currency_id,
COUNT(*) AS loan_count,
CAST(SUM(CASE WHEN l.repayment_status = 'active' THEN l.amount ELSE 0 END) AS decimal(18,2))
AS active_loan_principal,
CAST(SUM(CASE WHEN l.repayment_status = 'completed' THEN l.amount ELSE 0 END) AS
decimal(18,2)) AS completed_loan_principal,
CAST(SUM(CASE WHEN l.repayment_status = 'defaulted' THEN l.amount ELSE 0 END) AS
decimal(18,2)) AS defaulted_loan_principal
INTO #loan_agg
FROM loan.loan l
INNER JOIN account.account a
ON a.account_id = l.account_id
GROUP BY a.currency_id;

```

```

/* =====
ATM CASH BY CURRENCY
===== */
IF OBJECT_ID('tempdb..#atm_agg') IS NOT NULL DROP TABLE #atm_agg;

```

```

SELECT
ac.currency_id,
CAST(SUM(ac.amount) AS decimal(18,2)) AS atm_cash
INTO #atm_agg
FROM atm.atm_cash ac
GROUP BY ac.currency_id;

```

```

/* =====
1) EXECUTIVE SUMMARY
===== */
SELECT
@FromDate AS report_from,
@ToDate AS report_to,
(SELECT COUNT(*)
FROM customer.customer) AS total_customers,
(SELECT SUM(CASE WHEN status = 'active' THEN 1 ELSE 0 END)
FROM customer.customer) AS active_customers,
(SELECT COUNT(*)
FROM #tx) AS successful_transactions,
CAST((SELECT COALESCE(SUM(amount), 0)
FROM #tx) AS decimal(18,2)) AS successful_transaction_volume,
CAST((SELECT COALESCE(SUM(CASE WHEN transaction_type = 'deposit' THEN amount ELSE 0 END),
0)
FROM #tx) AS decimal(18,2)) AS deposit_volume,
CAST((SELECT COALESCE(SUM(CASE WHEN transaction_type = 'withdraw' THEN amount ELSE 0
END), 0)
FROM #tx) AS decimal(18,2)) AS withdraw_volume,
CAST((SELECT COALESCE(SUM(CASE WHEN transaction_type = 'transfer' THEN amount ELSE 0
END), 0)
FROM #tx) AS decimal(18,2)) AS transfer_volume,

```

```

CAST((SELECT COALESCE(SUM(CASE WHEN transaction_type = 'purchase' THEN amount ELSE 0
END), 0)
FROM #tx) AS decimal(18,2)) AS purchase_volume,
CAST((SELECT COALESCE(SUM(CASE WHEN transaction_type = 'bill_payment' THEN amount ELSE 0
END), 0)
FROM #tx) AS decimal(18,2)) AS bill_payment_volume,
CAST(
(SELECT COALESCE(SUM(CASE WHEN transaction_type = 'deposit' THEN amount ELSE 0 END), 0)
FROM #tx)
- (SELECT COALESCE(SUM(CASE WHEN transaction_type IN ('withdraw', 'purchase', 'bill_payment')
THEN amount ELSE 0 END), 0)
FROM #tx)
AS decimal(18,2)
) AS net_external_cash_flow;

```

```

/* =====

```

2) FINANCIAL POSITION BY CURRENCY

Note: currencies are not mixed because no FX table exists.

```

===== */

```

```

SELECT
c.currency_code,
c.currency_name,
ISNULL(a.total_accounts, 0) AS total_accounts,
ISNULL(a.active_accounts, 0) AS active_accounts,
ISNULL(a.blocked_accounts, 0) AS blocked_accounts,
ISNULL(a.dormant_accounts, 0) AS dormant_accounts,
ISNULL(a.closed_accounts, 0) AS closed_accounts,
CAST(ISNULL(a.total_customer_balances, 0) AS decimal(18,2)) AS customer_deposits_liability,
CAST(ISNULL(a.saving_balance, 0) AS decimal(18,2)) AS saving_balance,
CAST(ISNULL(a.current_balance, 0) AS decimal(18,2)) AS current_balance,
CAST(ISNULL(a.business_balance, 0) AS decimal(18,2)) AS business_balance,
CAST(ISNULL(a.loan_facility_balance, 0) AS decimal(18,2)) AS loan_facility_balance,
CAST(ISNULL(a.investment_balance, 0) AS decimal(18,2)) AS investment_balance,
ISNULL(l.loan_count, 0) AS loan_count,
CAST(ISNULL(l.active_loan_principal, 0) AS decimal(18,2)) AS active_loan_principal,
CAST(ISNULL(l.completed_loan_principal, 0) AS decimal(18,2)) AS completed_loan_principal,
CAST(ISNULL(l.defaulted_loan_principal, 0) AS decimal(18,2)) AS defaulted_loan_principal,
CAST(ISNULL(atm.atm_cash, 0) AS decimal(18,2)) AS atm_cash,
CAST(
ISNULL(atm.atm_cash, 0)
+ ISNULL(l.active_loan_principal, 0)
- ISNULL(a.total_customer_balances, 0)
AS decimal(18,2)
) AS simplified_net_position
FROM currency.currency c
LEFT JOIN #account_agg a ON a.currency_id = c.currency_id
LEFT JOIN #loan_agg l ON l.currency_id = c.currency_id
LEFT JOIN #atm_agg atm ON atm.currency_id = c.currency_id
ORDER BY c.currency_code;

```

```

/* =====
3) TRANSACTION BREAKDOWN
===== */
SELECT
transaction_type,
COUNT(*) AS transaction_count,
CAST(SUM(amount) AS decimal(18,2)) AS transaction_volume
FROM #tx
GROUP BY transaction_type
ORDER BY transaction_type;

/* =====
4) ACCOUNT TYPE BREAKDOWN
===== */
SELECT
CAST(SUM(saving_balance) AS decimal(18,2)) AS saving_total,
CAST(SUM(current_balance) AS decimal(18,2)) AS current_total,
CAST(SUM(business_balance) AS decimal(18,2)) AS business_total,
CAST(SUM(loan_facility_balance) AS decimal(18,2)) AS loan_facility_total,
CAST(SUM(investment_balance) AS decimal(18,2)) AS investment_total
FROM #account_agg;

/* =====
5) LOAN PORTFOLIO SUMMARY
===== */
SELECT
CAST(SUM(active_loan_principal) AS decimal(18,2)) AS active_loan_principal_total,
CAST(SUM(completed_loan_principal) AS decimal(18,2)) AS completed_loan_principal_total,
CAST(SUM(defaulted_loan_principal) AS decimal(18,2)) AS defaulted_loan_principal_total
FROM #loan_agg;

/* =====
6) ATM CASH SUMMARY
===== */
SELECT
CAST(SUM(atm_cash) AS decimal(18,2)) AS total_atm_cash
FROM #atm_agg;

```

(8) کلی منطق کسب و کار سیستم

این پایگاه داده فقط یک ساختار ذخیره سازی نیست، بلکه منطق بانک را مدل می کند.

چند نمونه از این منطق:

• مشتری می تواند حقیقی یا حقوقی باشد

- حساب می‌تواند چند مالک داشته باشد
- نوع تراکنش باید با ساختار آن سازگار باشد
- موجودی حساب باید قابل ردیابی باشد
- وضعیت تراکنش باید ثبت شود
- کارت به حساب وابسته است
- وام باید به حساب و ضامن متصل باشد
- چک‌بوک باید برای یک حساب و شعبه مشخص صادر شود
- دستگاه‌ها می‌توانند به مشتری متصل شوند
- اطلاعات حساس باید محافظت شوند
- دسترسی کاربران باید کنترل شود

این یعنی پایگاه داده بر اساس منطق واقعی یک سیستم بانکی طراحی شده و صرفاً مجموعه‌ای از جدول‌های مستقل نیست.

(9) جمع‌بندی نهایی

پایگاه داده طراحی شده، یک طراحی کامل و چندلایه برای یک سامانه بانکی است که هم از نظر **ساختار داده** و هم از نظر **منطق کسب‌وکار**، پیاده سازی شده است. در این طراحی:

- داده‌ها به صورت منطقی تفکیک شده‌اند
- روابط بین موجودیت‌ها دقیق تعریف شده‌اند
- محدودیت‌های مهم برای حفظ صحت داده اعمال شده‌اند
- امنیت با رمزنگاری و کنترل دسترسی تقویت شده است
- عملیات پرتکرار با ایندکس بهینه شده‌اند
- تاریخچه تغییرات مهم ثبت می‌شود
- گزارش‌های مدیریتی و تحلیلی پشتیبانی می‌شوند